KUTAISI INTERNATIONAL UNIVERSITY

Scientific Computing

Ramaz Botchorishvili

Kutaisi International University

2.1: Euler schemes: derivation, stability, implementation

Outline

Lecture 2.1: Euler schemes: derivation, stability, implementation

- Euler schemes from numerical differentiation and from numerical integration
- Explicit and implicit Euler schemes
- Stability on the test equation: A-stability
- Implementing implicit Euler: linear and nonlinear systems
- Q & A

Series: 2.1 Euler schemes. 2.2 errors. 2.3 geometry. 2.4 explicit convergence. 2.5 Grönwall. 2.6 implicit convergence.

Problem: the initial value problem and its grid

Problem 2.1 (Initial value problem $y' = f(t, y)$, $y(a) = y_a$, and time grid $t_n = a + nh$)

Find $y : [a, b] \to \mathbb{R}^n$ with

$$y'(t) = f(t, y(t)), \qquad y(a) = y_a,$$

where $f : [a, b] \times \mathbb{R}^n \to \mathbb{R}^n$.

Problem: the initial value problem and its grid

Problem 2.1 (Initial value problem $y' = f(t, y)$, $y(a) = y_a$, and time grid $t_n = a + nh$)

Find $y : [a, b] \to \mathbb{R}^n$ with

$$y'(t) = f(t, y(t)), \qquad y(a) = y_a,$$

where $f : [a, b] \times \mathbb{R}^n \to \mathbb{R}^n$.
Choose $m \in \mathbb{N}$ and the step size $h = (b - a)/m$. The grid is

$$t_n = a + nh, \qquad n = 0, 1, \ldots, m.$$

We look for numbers $y_n \approx y(t_n)$ with $y_0 = y_a$.

Euler scheme from numerical differentiation

Derivation from finite differences, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.

Euler scheme from numerical differentiation

Derivation from finite differences, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. The ODE holds at every grid point: $y'(t_n) = f(t_n, y(t_n))$.

Euler scheme from numerical differentiation

Derivation from finite differences, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. The ODE holds at every grid point: $y'(t_n) = f(t_n, y(t_n))$.
3. Replace the derivative by a forward difference:

$$\frac{y(t_{n+1}) - y(t_n)}{h} + O(h) = f(t_n, y(t_n)).$$

Euler scheme from numerical differentiation

Derivation from finite differences, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. The ODE holds at every grid point: $y'(t_n) = f(t_n, y(t_n))$.
3. Replace the derivative by a forward difference:

$$\frac{y(t_{n+1}) - y(t_n)}{h} + O(h) = f(t_n, y(t_n)).$$

4. Drop the truncation error $O(h)$ and write y_n for $y(t_n)$:

$$\frac{y_{n+1} - y_n}{h} = f(t_n, y_n).$$

Euler scheme from numerical differentiation

Derivation from finite differences, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. The ODE holds at every grid point: $y'(t_n) = f(t_n, y(t_n))$.
3. Replace the derivative by a forward difference:

$$\frac{y(t_{n+1}) - y(t_n)}{h} + O(h) = f(t_n, y(t_n)).$$

4. Drop the truncation error $O(h)$ and write y_n for $y(t_n)$:

$$\frac{y_{n+1} - y_n}{h} = f(t_n, y_n).$$

5. Check stability (next slides).

Euler scheme from numerical integration

Derivation from a quadrature rule, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.

Euler scheme from numerical integration

Derivation from a quadrature rule, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. Integrate the ODE over one step:
   $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$.

Euler scheme from numerical integration

Derivation from a quadrature rule, 5 steps

1. Grid: $t_n = a + nh$, $h = (b - a)/m$.
2. Integrate the ODE over one step:
   $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$.
3. Replace the integral by the left rectangle rule:

$$y(t_{n+1}) - y(t_n) = h\, f(t_n, y(t_n)) + O(h^2).$$

Euler scheme from numerical integration

Derivation from a quadrature rule, 5 steps

1. Grid: $t_n = a + nh$, $h = (b-a)/m$.
2. Integrate the ODE over one step:
   $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$.
3. Replace the integral by the left rectangle rule:

$$y(t_{n+1}) - y(t_n) = h\, f(t_n, y(t_n)) + O(h^2).$$

4. Drop the error $O(h^2)$:

$$y_{n+1} - y_n = h\, f(t_n, y_n).$$

Euler scheme from numerical integration

Derivation from a quadrature rule, 5 steps

1. Grid: $t_n = a + nh$, $h = (b-a)/m$.
2. Integrate the ODE over one step:
   $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$.
3. Replace the integral by the left rectangle rule:

$$y(t_{n+1}) - y(t_n) = h\, f(t_n, y(t_n)) + O(h^2).$$

4. Drop the error $O(h^2)$:

$$y_{n+1} - y_n = h\, f(t_n, y_n).$$

5. Check stability (next slides).

Explicit and implicit Euler schemes

Algorithm 2.2 (Explicit (forward) Euler scheme $y_{n+1} = y_n + hf(t_n, y_n)$)

$$y_{n+1} = y_n + h\, f(t_n, y_n), \qquad n = 0, 1, \ldots, m-1, \quad y_0 = y_a.$$

The new value y_{n+1} is computed directly from known values.

Explicit and implicit Euler schemes

Algorithm 2.2 (Explicit (forward) Euler scheme $y_{n+1} = y_n + hf(t_n, y_n)$)

$$y_{n+1} = y_n + h\, f(t_n, y_n), \qquad n = 0, 1, \ldots, m-1, \quad y_0 = y_a.$$

The new value y_{n+1} is computed directly from known values.

Algorithm 2.3 (Implicit (backward) Euler scheme $y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$)

$$y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1}), \qquad n = 0, 1, \ldots, m-1, \quad y_0 = y_a.$$

The unknown y_{n+1} appears on both sides: an equation has to be solved in every step. It comes from the backward difference $\frac{y_{n+1}-y_n}{h} = f(t_{n+1}, y_{n+1})$.

Definition: stability on the test equation

Definition 2.4 (A-stable, conditionally stable and unstable schemes on the test equation $y' = -ry$)

Apply a scheme to the test equation $y' = -ry$, $y(a) = y_a$, $r > 0$.

Definition: stability on the test equation

Definition 2.4 (A-stable, conditionally stable and unstable schemes on the test equation $y' = -ry$)

Apply a scheme to the test equation $y' = -ry$, $y(a) = y_a$, $r > 0$.

- If the numerical solution is bounded for every step size $h > 0$, the scheme is **A-stable**.

Definition: stability on the test equation

Definition 2.4 (A-stable, conditionally stable and unstable schemes on the test equation $y' = -ry$)

Apply a scheme to the test equation $y' = -ry$, $y(a) = y_a$, $r > 0$.

- If the numerical solution is bounded for every step size $h > 0$, the scheme is **A-stable**.
- If it is bounded only for small step sizes $h \leq h_0$, the scheme is **conditionally stable**.

Definition: stability on the test equation

Definition 2.4 (A-stable, conditionally stable and unstable schemes on the test equation $y' = -ry$)

Apply a scheme to the test equation $y' = -ry$, $y(a) = y_a$, $r > 0$.

- If the numerical solution is bounded for every step size $h > 0$, the scheme is **A-stable**.
- If it is bounded only for small step sizes $h \leq h_0$, the scheme is **conditionally stable**.
- If the numerical solution is unbounded, the scheme is **unstable**.

Definition: stability on the test equation

Definition 2.4 (A-stable, conditionally stable and unstable schemes on the test equation $y' = -ry$)

Apply a scheme to the test equation $y' = -ry$, $y(a) = y_a$, $r > 0$.

- If the numerical solution is bounded for every step size $h > 0$, the scheme is **A-stable**.
- If it is bounded only for small step sizes $h \leq h_0$, the scheme is **conditionally stable**.
- If the numerical solution is unbounded, the scheme is **unstable**.

Why this test

The exact solution $y_a e^{-r(t-a)}$ decays; a useful scheme must not blow up where the exact solution does not.

Proposition: stability of explicit Euler

Proposition 2.5 (Explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$)

For $y' = -ay$, $a > 0$, explicit Euler gives $y_n = (1 - ah)^n y_0$. The sequence is bounded if and only if

$$|1 - ah| \leq 1 \iff h \leq \frac{2}{a}.$$

For $h < 1/a$ the factor $1 - ah$ is positive and the numerical solution decays monotonically, like the exact one.

Proposition: stability of explicit Euler

Proposition 2.5 (Explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$)

For $y' = -ay$, $a > 0$, explicit Euler gives $y_n = (1 - ah)^n y_0$. The sequence is bounded if and only if

$$|1 - ah| \leq 1 \iff h \leq \frac{2}{a}.$$

For $h < 1/a$ the factor $1 - ah$ is positive and the numerical solution decays monotonically, like the exact one.

Proof that explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$.

- Scheme: $\frac{y_{n+1} - y_n}{h} = -ay_n \Rightarrow y_{n+1} = (1 - ah)y_n$.

Proposition: stability of explicit Euler

Proposition 2.5 (Explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$)

For $y' = -ay$, $a > 0$, explicit Euler gives $y_n = (1 - ah)^n y_0$. The sequence is bounded if and only if

$$|1 - ah| \leq 1 \iff h \leq \frac{2}{a}.$$

For $h < 1/a$ the factor $1 - ah$ is positive and the numerical solution decays monotonically, like the exact one.

Proof that explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$.

- Scheme: $\frac{y_{n+1} - y_n}{h} = -ay_n \Rightarrow y_{n+1} = (1 - ah)y_n$.
- Repeating: $y_n = (1 - ah)^n y_0$.

Proposition: stability of explicit Euler

Proposition 2.5 (Explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$)

For $y' = -ay$, $a > 0$, explicit Euler gives $y_n = (1 - ah)^n y_0$. The sequence is bounded if and only if

$$|1 - ah| \leq 1 \iff h \leq \frac{2}{a}.$$

For $h < 1/a$ the factor $1 - ah$ is positive and the numerical solution decays monotonically, like the exact one.

Proof that explicit Euler on $y' = -ay$ is stable if and only if $h \leq 2/a$.

- Scheme: $\frac{y_{n+1} - y_n}{h} = -ay_n \Rightarrow y_{n+1} = (1 - ah)y_n$.
- Repeating: $y_n = (1 - ah)^n y_0$.
- $|1 - ah| \leq 1$ gives $|y_n| \leq |y_0|$; $|1 - ah| > 1$ gives $|y_n| \to \infty$ when $y_0 \neq 0$.

Proposition: stability of implicit Euler

Proposition 2.6 (Implicit Euler on $y' = -ay$ is stable for every $h > 0$ (A-stable))

For $y' = -ay$, $a > 0$, implicit Euler gives

$$y_n = \frac{y_0}{(1+ah)^n}, \qquad |y_n| \le |y_0| \quad \textit{for every } h > 0.$$

Proposition: stability of implicit Euler

Proposition 2.6 (Implicit Euler on $y' = -ay$ is stable for every $h > 0$ (A-stable))

For $y' = -ay$, $a > 0$, implicit Euler gives

$$y_n = \frac{y_0}{(1+ah)^n}, \qquad |y_n| \le |y_0| \quad \textit{for every } h > 0.$$

Proof that implicit Euler on $y' = -ay$ is A-stable.

- Scheme: $\frac{y_{n+1}-y_n}{h} = -ay_{n+1} \;\Rightarrow\; (1+ah)\, y_{n+1} = y_n.$

Proposition: stability of implicit Euler

Proposition 2.6 (Implicit Euler on $y' = -ay$ is stable for every $h > 0$ (A-stable))

For $y' = -ay$, $a > 0$, implicit Euler gives

$$y_n = \frac{y_0}{(1+ah)^n}, \qquad |y_n| \le |y_0| \quad \textit{for every } h > 0.$$

Proof that implicit Euler on $y' = -ay$ is A-stable.

- Scheme: $\frac{y_{n+1}-y_n}{h} = -ay_{n+1} \;\Rightarrow\; (1+ah)\, y_{n+1} = y_n$.
- Hence $y_{n+1} = \frac{1}{1+ah}\, y_n$ and $0 < \frac{1}{1+ah} < 1$ for all $h > 0$.

Proposition: stability of implicit Euler

Proposition 2.6 (Implicit Euler on $y' = -ay$ is stable for every $h > 0$ (A-stable))

For $y' = -ay$, $a > 0$, implicit Euler gives

$$y_n = \frac{y_0}{(1+ah)^n}, \qquad |y_n| \le |y_0| \quad \textit{for every } h > 0.$$

Proof that implicit Euler on $y' = -ay$ is A-stable.

- Scheme: $\frac{y_{n+1}-y_n}{h} = -ay_{n+1} \;\Rightarrow\; (1+ah)\, y_{n+1} = y_n$.
- Hence $y_{n+1} = \frac{1}{1+ah}\, y_n$ and $0 < \frac{1}{1+ah} < 1$ for all $h > 0$.
- Thus $y_n = y_0/(1+ah)^n$ is bounded for every step size.

□

Problem: implementing implicit Euler, linear systems

Problem 2.7 (Solving the implicit Euler step $(I - hA)y_{n+1} = y_n$ for the linear system $u' = Au$)

$u'(t) = Au$, $u(0) = u_0$, $A \in \mathbb{R}^{n \times n}$, $u : [0, T] \to \mathbb{R}^n$.

Problem: implementing implicit Euler, linear systems

Problem 2.7 (Solving the implicit Euler step $(I - hA)y_{n+1} = y_n$ for the linear system $u' = Au$)

$u'(t) = Au$, $u(0) = u_0$, $A \in \mathbb{R}^{n\times n}$, $u : [0, T] \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = Ay_{n+1}$, that is

$$(I - hA)\, y_{n+1} = y_n \qquad \text{a linear system of algebraic equations.}$$

Problem: implementing implicit Euler, linear systems

Problem 2.7 (Solving the implicit Euler step $(I - hA)y_{n+1} = y_n$ for the linear system $u' = Au$)

$u'(t) = Au$, $u(0) = u_0$, $A \in \mathbb{R}^{n\times n}$, $u : [0, T] \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = Ay_{n+1}$, that is

$$(I - hA)\, y_{n+1} = y_n \qquad \text{a linear system of algebraic equations.}$$

How shall we solve the system?

- $y_{n+1} = (I - hA)^{-1} y_n$: not practical, expensive and accumulates errors.

Problem: implementing implicit Euler, linear systems

Problem 2.7 (Solving the implicit Euler step $(I - hA)y_{n+1} = y_n$ for the linear system $u' = Au$)

$u'(t) = Au$, $u(0) = u_0$, $A \in \mathbb{R}^{n\times n}$, $u : [0, T] \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = Ay_{n+1}$, that is

$$(I - hA)\, y_{n+1} = y_n \qquad \text{a linear system of algebraic equations.}$$

How shall we solve the system?

- $y_{n+1} = (I - hA)^{-1} y_n$: not practical, expensive and accumulates errors.
- Direct solvers (Gaussian elimination): expensive for large n, especially when $A = A(t)$ changes in time.

Problem: implementing implicit Euler, linear systems

Problem 2.7 (Solving the implicit Euler step $(I - hA)y_{n+1} = y_n$ for the linear system $u' = Au$)

$u'(t) = Au$, $u(0) = u_0$, $A \in \mathbb{R}^{n \times n}$, $u : [0, T] \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = Ay_{n+1}$, that is

$$(I - hA)\, y_{n+1} = y_n \qquad \text{a linear system of algebraic equations.}$$

How shall we solve the system?

- $y_{n+1} = (I - hA)^{-1} y_n$: not practical, expensive and accumulates errors.
- Direct solvers (Gaussian elimination): expensive for large n, especially when $A = A(t)$ changes in time.
- Other option: iterative solvers.

Problem: implementing implicit Euler, nonlinear systems

Problem 2.8 (Solving the implicit Euler step for the nonlinear system $u' = F(t, u)$)

$u'(t) = F(t, u)$, $u(0) = u_0$, $F : [0, T] \times \mathbb{R}^n \to \mathbb{R}^n$.

Problem: implementing implicit Euler, nonlinear systems

Problem 2.8 (Solving the implicit Euler step for the nonlinear system $u' = F(t, u)$)

$u'(t) = F(t, u)$, $u(0) = u_0$, $F : [0, T] \times \mathbb{R}^n \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = F(t_{n+1}, y_{n+1})$, that is

$y_{n+1} - hF(t_{n+1}, y_{n+1}) = y_n$ a nonlinear system of algebraic equations.

Problem: implementing implicit Euler, nonlinear systems

Problem 2.8 (Solving the implicit Euler step for the nonlinear system $u' = F(t, u)$)

$u'(t) = F(t, u)$, $u(0) = u_0$, $F : [0, T] \times \mathbb{R}^n \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = F(t_{n+1}, y_{n+1})$, that is

$y_{n+1} - hF(t_{n+1}, y_{n+1}) = y_n$ a nonlinear system of algebraic equations.

How shall we solve the system?

- No direct methods are available in general.

Problem: implementing implicit Euler, nonlinear systems

Problem 2.8 (Solving the implicit Euler step for the nonlinear system $u' = F(t, u)$)

$u'(t) = F(t, u)$, $u(0) = u_0$, $F : [0, T] \times \mathbb{R}^n \to \mathbb{R}^n$.

Implicit Euler: $\dfrac{y_{n+1} - y_n}{h} = F(t_{n+1}, y_{n+1})$, that is

$y_{n+1} - hF(t_{n+1}, y_{n+1}) = y_n$ a nonlinear system of algebraic equations.

How shall we solve the system?

- No direct methods are available in general.
- Approach: iterative methods for nonlinear equations, started from y_n.

Remark: explicit versus implicit Euler

Remark 2.9 (Explicit versus implicit Euler: cost per step against stability)

	explicit Euler	implicit Euler
work per step	one evaluation of f	solve a (non)linear system
stability on $y' = -ay$	only for $h \leq 2/a$	every $h > 0$
order (shown later)	1	1

The extra work per step buys stability; this matters for stiff problems.

Q & A